ADVANCED JAVA

(3160707)

LABORATORY MANUAL

B.E. Semester-VI

Computer Engineering

Prepared By:-
CE/IT Department
Vadodara Institute of Engineering (080)
Kotambi, Vadodara – 391510
Academic Year : 2025-2026

Index

Software Requirements

Sr No	Software Requirement	Hardware Requirement
1	Visual studio code	64-bit OS
		RAM-16 GB
		Processor (Intel i5/i7, AMD Ryzen, etc.)
		Hard Disk- 512 GB

Vadodara Institute of Engineering (080)
Advance java Programming (3160707)

Date:

PRACTICAL - 1

TCP One way communication

Aim: Create an application using TCP to implement one way communication.

Theory:

- Java Socket programming is used for communication between the applications running on different JRE.
- Java Socket programming can be connection-oriented or connection-less.
- Socket and ServerSocket classes are used for connection-oriented socket programming and DatagramSocket and DatagramPacket classes are used for connection-less socket programming.

Syntax:

```
ServerSocket ss=new ServerSocket(6666);
Socket s=ss.accept();//establishes connection
DataInputStream dis=new DataInputStream(s.getInputStream());
```

Flowchart/Figure/Algorithm:

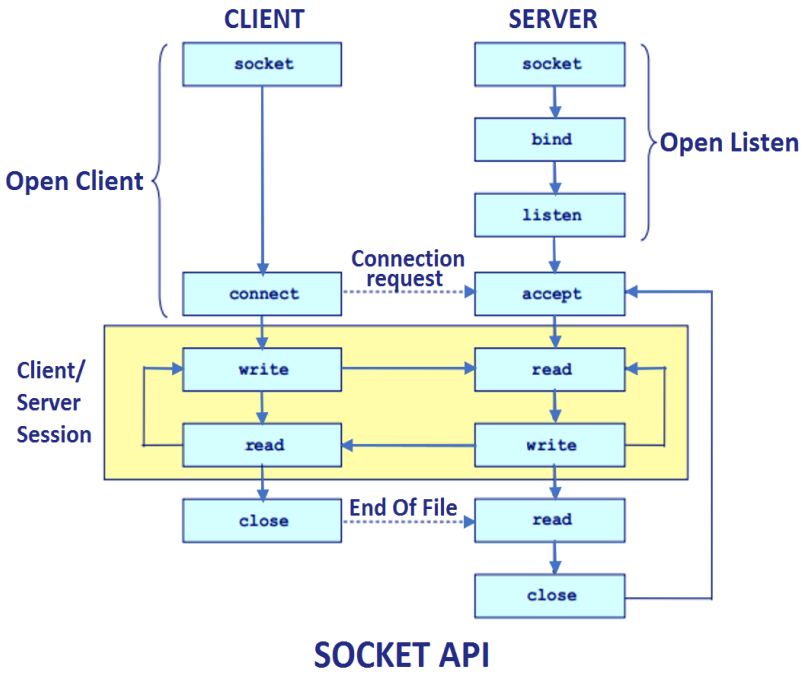


Fig.1: Flowchart of Socket Programming

Program/Implementation:

SenderServer.java

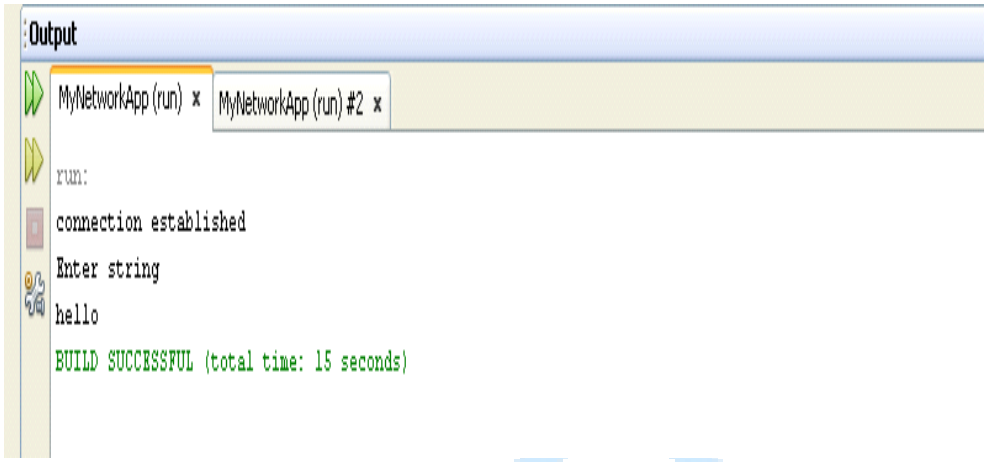
```
package mynetworkapp;
import java.io.*;

import java.net.*;
import java.util.Scanner;

public class SenderServer {


   public static void main(String[] args) {
      // TODO code application logic here
      try{
         ServerSocket ss=new ServerSocket(4020);
         System.out.println("wating for client to connect");
         Socket s=ss.accept();

         System.out.println("connection established");

         PrintStream ps=new PrintStream(s.getOutputStream());
         BufferedReader kb=new BufferedReader(new InputStreamReader(System.in));

         System.out.println("Enter data :");
         String str=kb.readLine();

         while(!str.equals("exit"))
         {   ps.println(str);
            System.out.println("Enter data :");
            str=kb.readLine();
         }
         ps.close();
         s.close();
         ss.close();

      }catch(IOException e)
      {
         System.out.println(e);
      }
   }
}
```

ReceiverClient.java

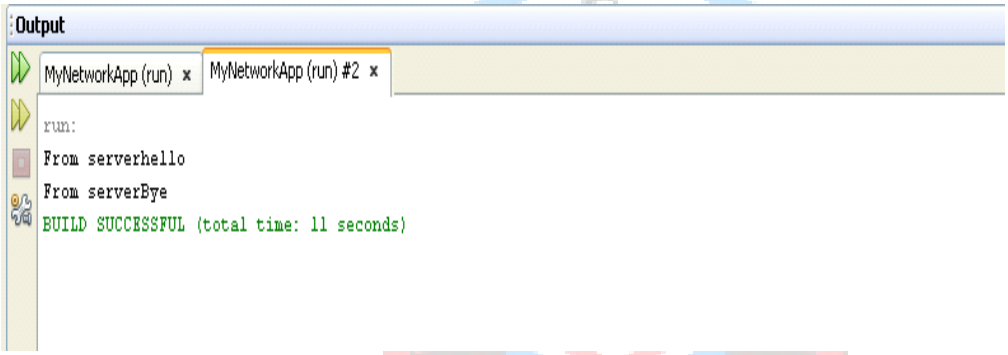
```
package mynetworkapp;

import java.net.InetAddress;
import java.net.Socket;
import java.util.*;
import java.io.*;

public class ReceiverClient {
   public static void main(String[] args) {
      // TODO code application logic here
      try{
        // InetAddress ip=InetAddress.getLocalHost();
         Socket s=new Socket("localhost",4020);

         BufferedReader br=new BufferedReader(new InputStreamReader(s.getInputStream()));

         String str;

         str=br.readLine();

         while(str!=null)
         {
            System.out.println(str);
            str=br.readLine();
         }
         br.close();
         s.close();
      }catch(Exception e)
      {
         System.out.println(e);
      }

   }
}
```

Output**:**

Server Side



Client Side



Logical Applications*:*

1. How to establish connection between client and server?

   _____

   _____

   _____

2. How to send message from server to client only?

   _____

   _____

   _____

Date:

PRACTICAL -2

TCP Two way communication

Aim: Create an application using TCP to implement two way communication.

Theory:

- Java Socket programming is used for communication between the applications running on different JRE.
- Java Socket programming can be connection-oriented or connection-less.
- Socket and ServerSocket classes are used for connection-oriented socket programming and DatagramSocket and DatagramPacket classes are used for connection-less socket programming.

Syntax:

```
ServerSocket ss=new ServerSocket(6666);
Socket s=ss.accept();//establishes connection
DataInputStream dis=new DataInputStream(s.getInputStream());
DataOutputStream dout=new DataOutputStream(s.getOutputStream())
```

Flowchart/Figure/Algorithm:

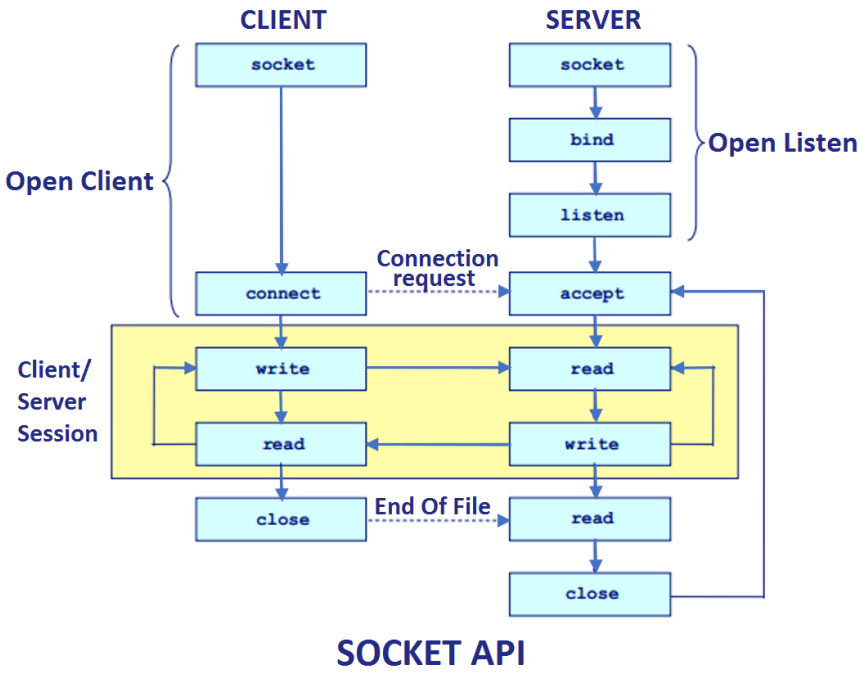


Fig.1: Flowchart of Socket Programming

Program/Implementation:

Server.java

```
package mynetworkapp;

import java.io.*;

import java.net.*;

public class TwoWayServer1 {

    public static void main(String[] args) {

     try{
       ServerSocket ss=new ServerSocket(4021);
       System.out.println("Wating for client to connect");
       Socket s=ss.accept();

       System.out.println("connection established");

       BufferedReader br=new BufferedReader(new InputStreamReader(s.getInputStream()));
       BufferedReader kb=new BufferedReader(new InputStreamReader(System.in));
       PrintStream ps=new PrintStream(s.getOutputStream());

       String str,str1;
       str=br.readLine();//read frm client

        while(str!=null)
         {
            System.out.println(str);//reads frm client n writes on server
            System.out.print("Write something  : ");
            str1=kb.readLine();
            ps.println(str1);

            str=br.readLine();//reads frm client
         }
       ps.close();
       br.close();
       kb.close();
       s.close();
       ss.close();
      }catch(IOException e)

    {
```

```
            System.out.println(e);
        }
    }
}
```

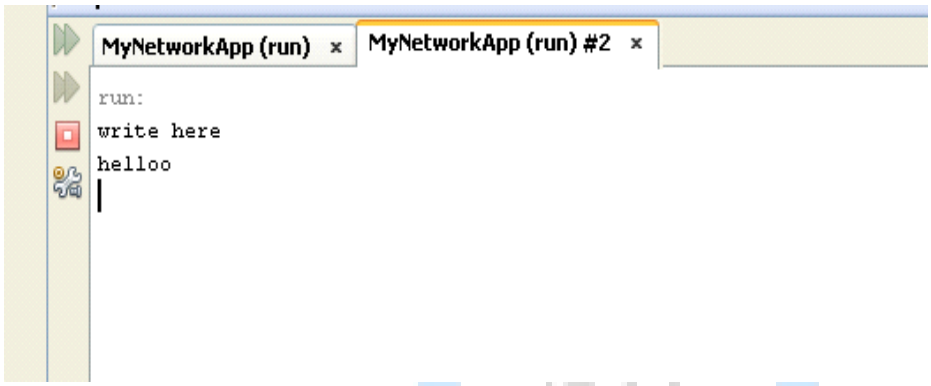
Client.java

```
package mynetworkapp;

import java.io.*;
import java.net.*;

public class TwoWayClient1 {


    public static void main(String[] args) {

        try{
            Socket s=new Socket("192.168.1.46",4021);

            BufferedReader br=new BufferedReader(new InputStreamReader(s.getInputStream()));
            BufferedReader kb=new BufferedReader(new InputStreamReader(System.in));
            PrintStream ps=new PrintStream(s.getOutputStream());

            System.out.print("write something : ");
            String str=kb.readLine()

            String str1;

              while(!(str.equals("exit")))
              {
                  ps.println(str);

                  str1=br.readLine();
                  System.out.println(str1);
                  System.out.print("write something : ");
                  str=kb.readLine();
              }
            ps.close();
            br.close();
            kb.close();
            s.close();
            System.out.println("client program ended");

        }catch(IOException e)
        {
            System.out.println(e);
```

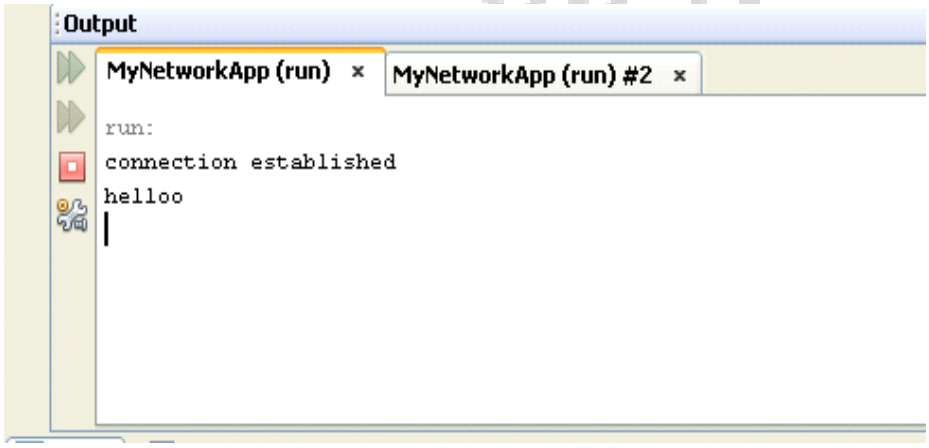
```
        }
    }
}
```

Output:

Client Side



Server side



Logical Applications*:*

1. Write a program in which server sends a string and client reply with reverse of that string?

   _____
   _____
   _____

2. Write a client server program using TCP where client sends astring and server checks whether that string is palindrome or not and responds with appropriate message.

   _____
   _____
   _____

Date:

PRACTICAL - 3

FileTransfer

Aim: Implement TCP Server for transferring files using Socket and ServerSocket.

Theory: File Transfer

- Client can send the file name to the server to access that file.
- After sending the file name by client,server checks that the file is available or not.If it is available than server sends the contents of that file to the client.

Syntax:

File f=new File(fname);

Flowchart/Figure/Algorithm:

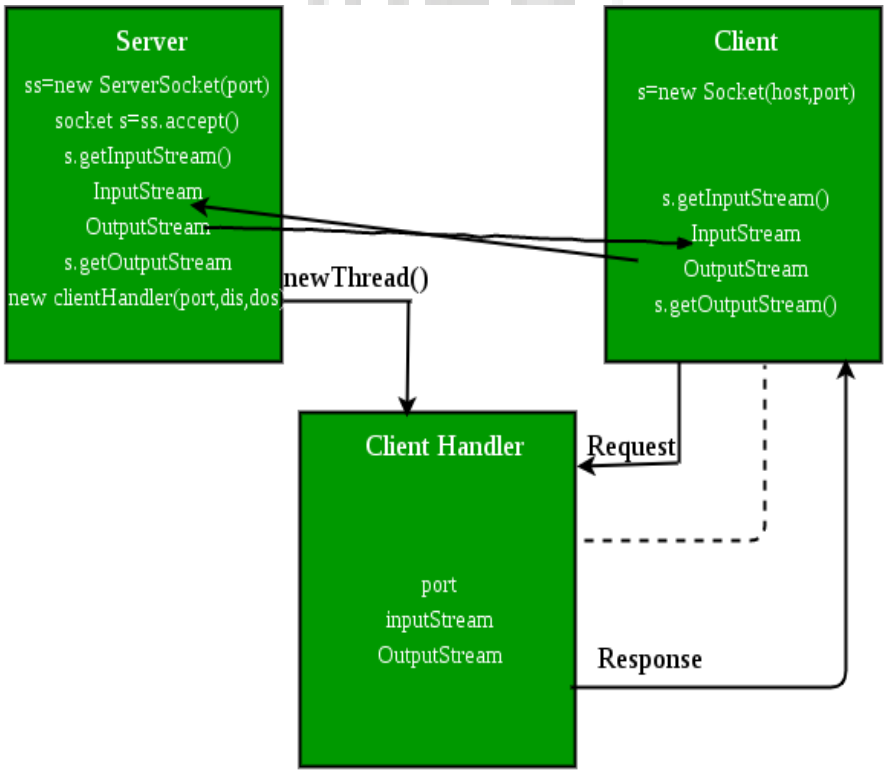


Fig 2. Flowchart for File Transfer

Program/Implementation:

FileServer.java

```
package mynetworkapp;

import java.io.*;

import java.net.*;

public class FileServer {

   public static void main(String[] args) {

      try{
        ServerSocket ss=new ServerSocket(4022);
        System.out.println("Wating for client to connect");
        Socket s=ss.accept();

        System.out.println("connection established");

        BufferedReader br=new BufferedReader(new InputStreamReader(s.getInputStream()));

        PrintStream ps=new PrintStream(s.getOutputStream());//client

        String fname;
        fname=br.readLine();

        File f=new File(fname);
        if(f.exists()==true)
        {
        BufferedReader br1=new BufferedReader(new FileReader(fname));
        String str;
        str=br1.readLine();
        while(str!=null)
        {
           ps.println(str);
           str=br1.readLine();
        }
         br1.close();
        }
        else
        {
           ps.println("file doesnt exist");
        }
```

```
            ps.close();
            br.close();
            s.close();
            ss.close();

        }catch(IOException e)

        {
            System.out.println(e);
        }
    }
}
```

FileClient.java

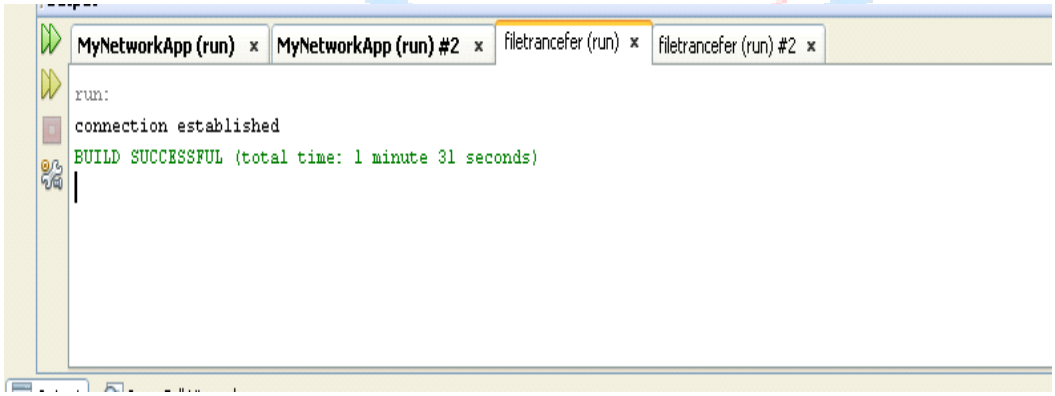
```
package mynetworkapp;

import java.io.*;
import java.net.InetAddress;
import java.net.Socket;

public class FileClient {

    public static void main(String[] args) {

        try{
            Socket s=new Socket("localhost",4022);

            BufferedReader br=new BufferedReader(new InputStreamReader(s.getInputStream()));
            BufferedReader kb=new BufferedReader(new InputStreamReader(System.in));
            PrintStream ps=new PrintStream(s.getOutputStream());

            System.out.print("Enter file name : ");
            String str=kb.readLine();
            ps.println(str);

             String st=br.readLine();
             while(st!=null)
             {
                System.out.println(st);
                st=br.readLine();
             }
            ps.close();
            br.close();
            kb.close();
            s.close();
```

```
            System.out.println("client program ended");
        }
         catch(IOException e)
        {
            System.out.println(e);
        }
    }
}
```

Output**:**

Client Side

Server Side



Logical Applications*:*

1. Write a TCP program to access the contents of file at client side.

   _____
   _____
   _____

2. Which classes are used for file transfer in socket programming?

   _____
   _____
   _____

Date:

PRACTICAL - 4

Statement and prepared Statement

Aim: User can create a new database and also create new table under that database. Once the database has been created then user can perform database operation by using Statement Interface.

Theory: Statement

- The **Statement interface** provides methods to execute queries with the database. The statement interface is a factory of ResultSet i.e. it provides factory method to get the object of ResultSet.
- The PreparedStatement interface is a subinterface of Statement. It is used to execute parameterized query.

Syntax:

```
Statement stmt=con.createStatement();
PreparedStatement stmt=con.prepareStatement("insert into Emp values(?,?)");
```

Flowchart/Figure/Algorithm:

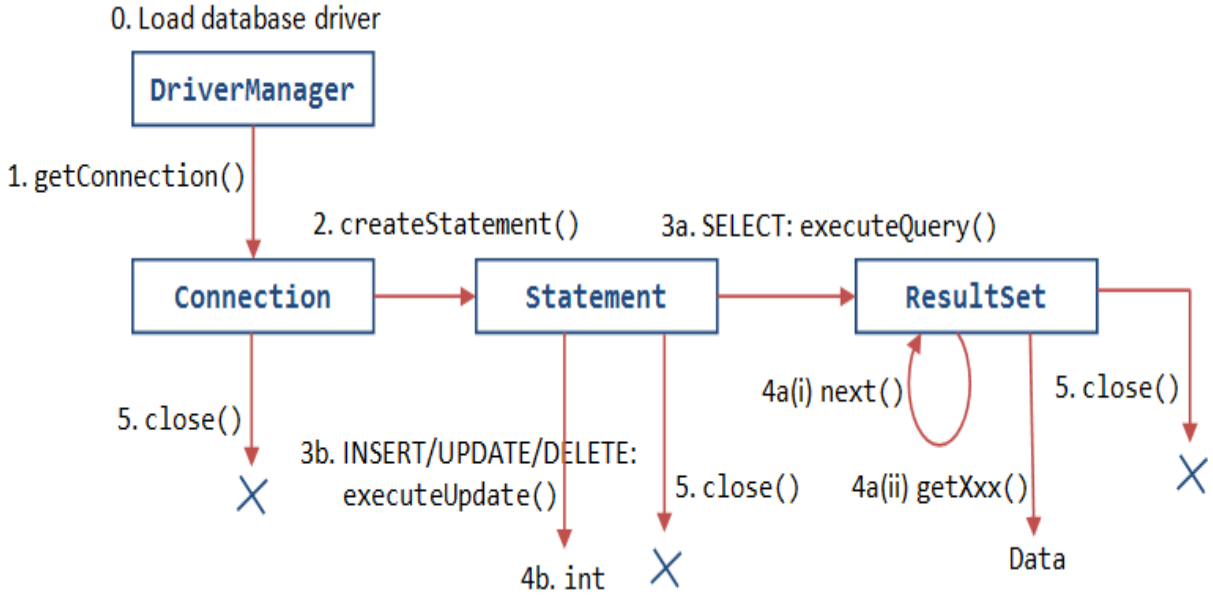
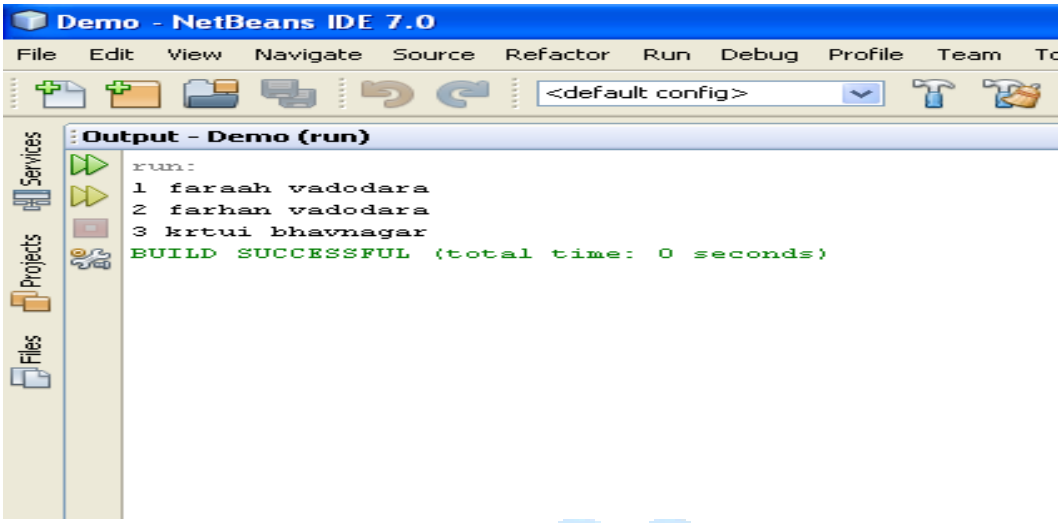


Fig.1: Flowchart of statement and preparedstatement interface

Program/Implementation:

```
package myfirstapp;
import java.sql.*;

public class JDBCStatement {

  public static void main(String[] args) {

       try
       {
          Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");

          Connection
con=DriverManager.getConnection("jdbc:odbc:TempDataSource","system","system");
          Statement st=con.createStatement();

          ResultSet rs=st.executeQuery("select * from student");

          while(rs.next())
          {

             System.out.println(rs.getString("id") + " "+rs.getString("s_name")+ " "+
rs.getString("address"));

          }

       }catch(SQLException e)
       {
         System.out.print(e);
       }
      catch(ClassNotFoundException e)
       {
         System.out.print(e);
       }

    }

}
```

Output:



Logical Applications:

1. Write a TCP program to access the contents of file at client side.

_____

_____

_____

2. Which classes are used for file transfer in socket programming?

_____

_____

_____

Date:

PRACTICAL - 5

Callable Statement

Aim: Implement Student information system using JDBC Callable Statement.

Theory: Callable Satement

- Callable Statement interface is used to call the stored procedures and functions.
- We can have business logic on the database by the use of stored procedures and functions that will make the performance better because these are precompiled.

Syntax:

```
CallableStatement stmt=con.prepareCall("{call myprocedure(?,?)}");
```

Flowchart/Figure/Algorithm:

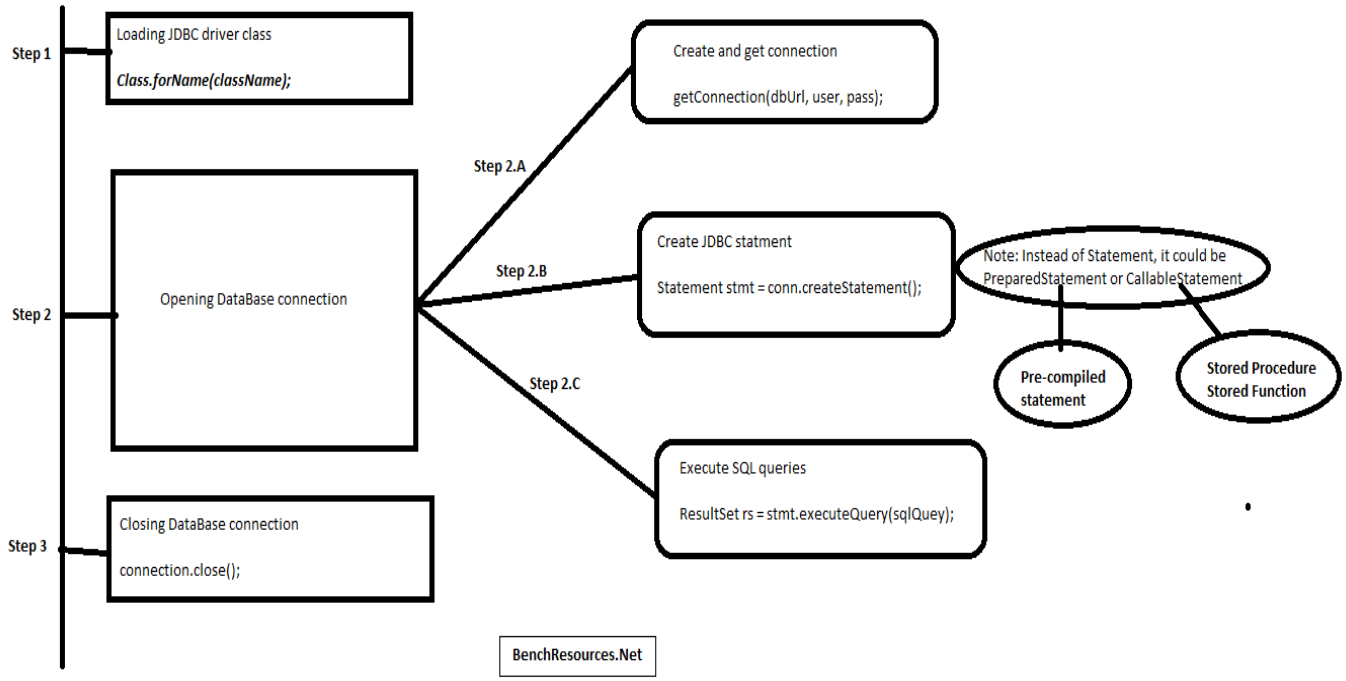


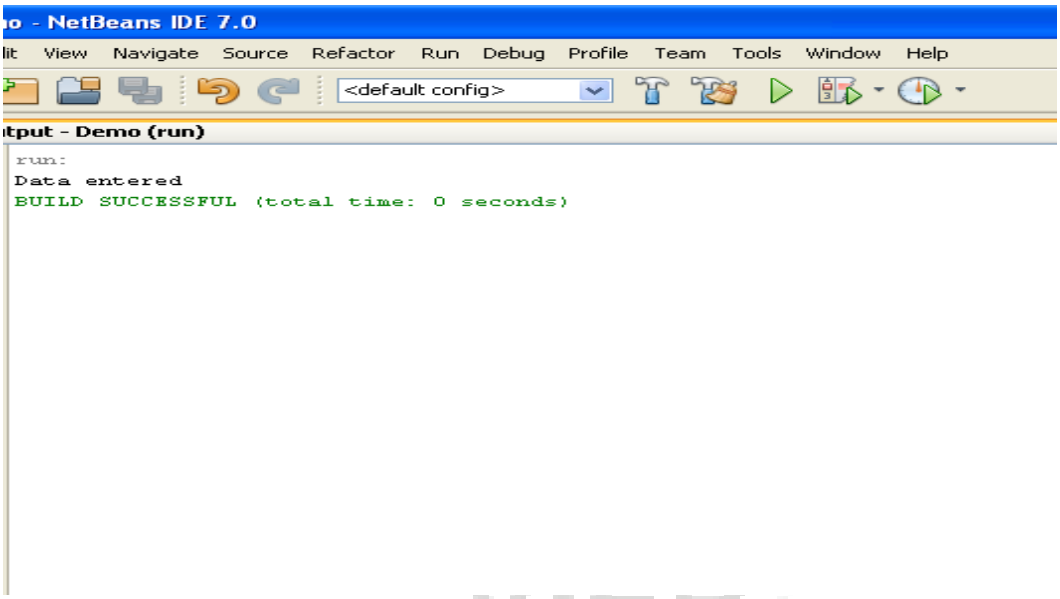
Fig 2. Flowchart for CallableStatement Interface

Program/Implementation:

```
package myfirstapp;

import java.sql.*;
import java.util.Scanner;

public class JDBCCallableStatement {

   public static void main(String[] args) {

      try
      {
         String name="vrushil11";
         int enr=13;
         int sem=5;
         String add="kotambi";
         int per=99;

         Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");

         Connection
con=DriverManager.getConnection("jdbc:odbc:TempDataSource","system","system");

         CallableStatement cs=con.prepareCall("call add_student_details (?,?,?,?,?)");

         cs.setInt(1, enr);

         cs.setString(2, name);
         cs.setInt(3,sem);
         cs.setString(4, add);
         cs.setInt(5, per);

         cs.execute();

         System.out.println("Data entered");

      }
     catch(SQLException e)
      {

        System.out.print(e);
      }
     catch(ClassNotFoundException e)
      {
```

```
            System.out.print(e);
        }
    }
}
```

Output:



Logical Applications:

1. What is Stored Procedure?

   _____

   _____

   _____

2. How to execute more than one query by using callable statement interface?

   _____

   _____

   _____

3. UpDate the record in the database by using callable statement interface.

   _____

   _____

   _____

Date:

PRACTICAL - 6

ServletContex and ServletConfig

Aim: Create a Servlet and retrieve Context and Config parameters.

Theory: ServletContex and ServletConfig

- An object of ServletConfig is created by the web container for each servlet. This object can be used to get configuration information from web.xml file.
- An object of ServletContext is created by the web container at time of deploying the project. This object can be used to get configuration information from web.xml file. There is only one ServletContext object per web application.

Syntax:

```
ServletConfig config=getServletConfig();
ServletContext application=getServletConfig().getServletContext();
```

Flowchart/Figure/Algorithm:

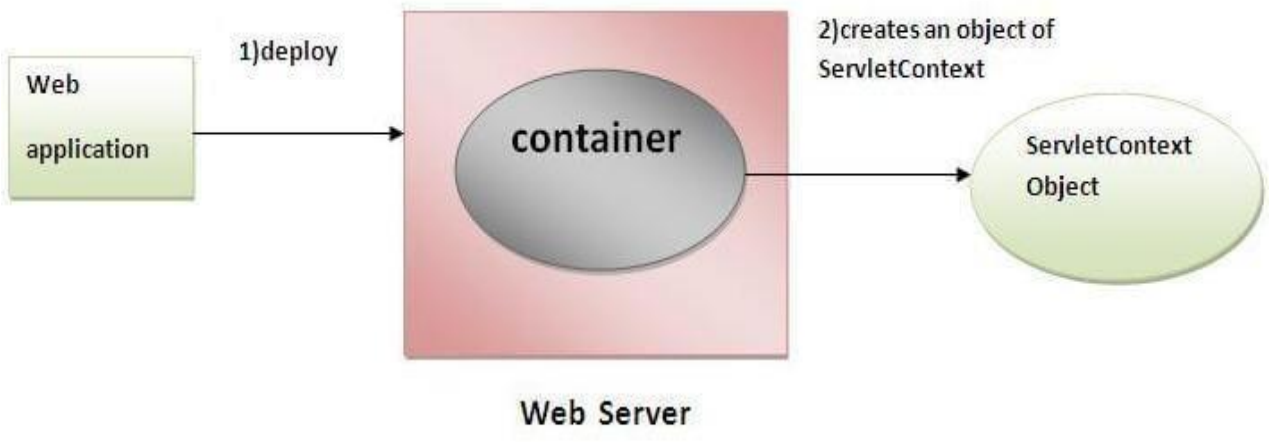


Fig 2. Flowchart for ServletContex

Program/Implementation:

index.jsp

```
<%--
   Document   : index
   Created on : Feb 16, 2016, 10:37:58 AM
   Author     : faraahdabhoiwala
--%>

<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
   <head>
      <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
      <title>JSP Page</title>
   </head>
   <body>
      <a href="RetriveParam"> Go to MyServlet</a>
   </body>
</html>
```

WEB-INF

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app version="3.0" xmlns="http://java.sun.com/xml/ns/javaee"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://java.sun.com/xml/ns/javaee http://java.sun.com/xml/ns/javaee/web-
app_3_0.xsd">
   <context-param>
      <param-name>admin</param-name>
      <param-value>dhaval</param-value>
   </context-param>
   <context-param>
      <param-name>user</param-name>
      <param-value>saifali</param-value>
   </context-param>
   <servlet>
      <servlet-name>RetriveParam</servlet-name>
      <servlet-class>RetriveParam</servlet-class>
      <init-param>
         <param-name>country</param-name>
         <param-value>india</param-value>
      </init-param>
      <init-param>
         <param-name>college</param-name>
```

```
            <param-value>vier</param-value>
        </init-param>
        <init-param>
            <param-name>s_name</param-name>
            <param-value>nirlay</param-value>
        </init-param>
    </servlet>
    <servlet-mapping>
        <servlet-name>RetriveParam</servlet-name>
        <url-pattern>/RetriveParam</url-pattern>
    </servlet-mapping>
    <session-config>
        <session-timeout>
            30
        </session-timeout>
    </session-config>
</web-app>
```

RetriveParam.java

```
import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletConfig;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.*;
import java.util.*;

public class RetriveParam extends HttpServlet {

      protected void processRequest(HttpServletRequest request, HttpServletResponse response)
          throws ServletException, IOException {
       response.setContentType("text/html;charset=UTF-8");
       PrintWriter out = response.getWriter();
        ServletConfig sc=this.getServletConfig();

        Enumeration e=sc.getInitParameterNames();
        while(e.hasMoreElements())
        {
            String str=(String)e.nextElement();//name of param
            out.print("<br><b>"+sc.getInitParameter(str));//value
        }
       ServletContext sc1=this.getServletContext();
        Enumeration e1=sc1.getInitParameterNames();
```

```
        while(e1.hasMoreElements())
        {
            String str1=(String)e1.nextElement();//name of param
            out.print("<br>"+str1);
            out.print("<br><b>"+sc1.getInitParameter(str1));
        }

    }

   @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        processRequest(request, response);
    }
    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        processRequest(request, response);
    }
```

OUTPUT:

dhaval
saifali
india
vier
nirlay

Logical Applications*:*

1. Write a program for getting all the initialization parameter from the web.xml file and printing this information in the servlet.

   _____

   _____

   _____

2. Which methods are used for ServletContext and ServletConfig?

   _____

   _____

   _____

Date:

PRACTICAL - 7

Servlet with database

Aim: Create Servlet file to insert, up Date **and delete records of particular table of database.**

Theory: Servlet with database

- To access the database by using servlet,within a servlet file we can use JDBC code for the same.

Syntax:

```
ServletConfig config=getServletConfig();
ServletContext application=getServletConfig().getServletContext();
```

Flowchart/Figure/Algorithm:

ss

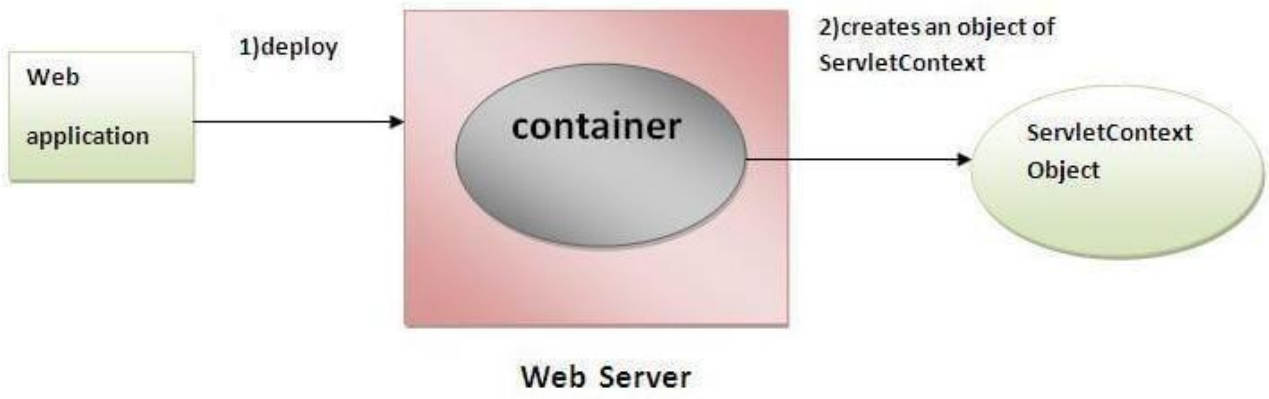


Fig 2. Flowchart for ServletContex

Program/Implementation:

Student.html

```
<!--
To change this template, choose Tools | Templates
and open the template in the editor.
-->
<!DOCTYPE html>
<html>
   <head>
      <title></title>
      <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
   </head>
   <body>

      <form action="JDBCServlet">
      Name<input type="text" name="name"></input>
      <br>
      Enrollment NO<input type="text" name="enr"></input>
      <br>
      Semester<input type="text" name="sem"></input>
      <br>
      <input type="submit" value="insert" name="submit"></input>
      <input type="submit" value="upDate" name="submit"></input>
        <input type="submit" value="delete" name="submit"></input>
      </form>
   </body>
</html>
```

JDBCServlet.java

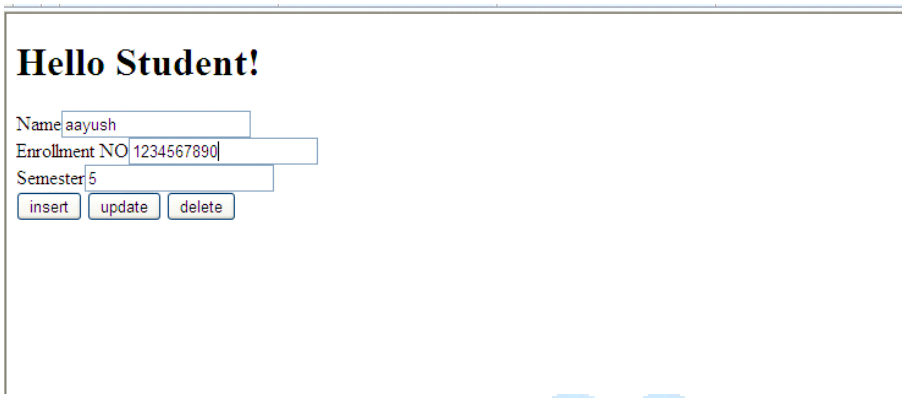
```
import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import java.sql.*;

public class JDBCServlet extends HttpServlet {
    int enr,sem;
    String name;
    protected void processRequest(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        response.setContentType("text/html;charset=UTF-8");
```

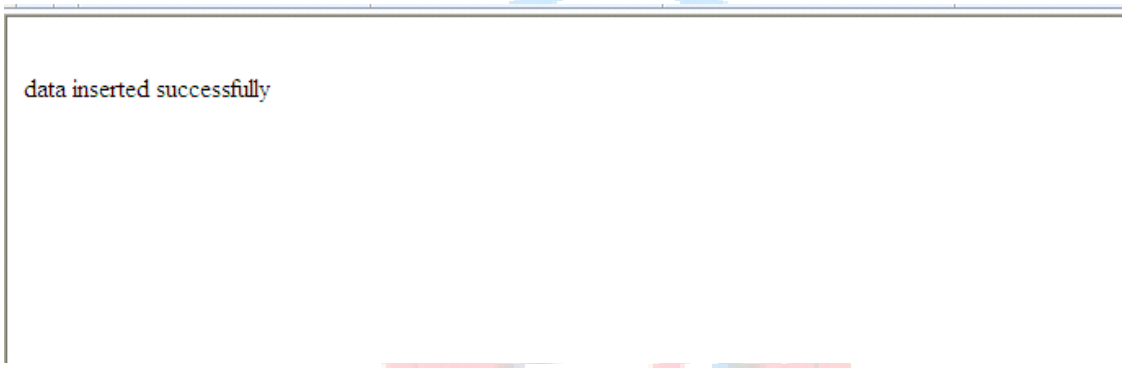
```
        PrintWriter out = response.getWriter();

         enr=Integer.parseInt(request.getParameter("enr"));
         name=request.getParameter("name");
         sem=Integer.parseInt(request.getParameter("sem"));

        String str=request.getParameter("submit");
        if(str.equals("insert"))
        {
            String s=insertData();
            out.println("<br> "+s);
        }
        else if(str.equals("upDate"))
        {
            upDateData();
             out.println("<br> data upDated successfully");
        }
        else if(str.equals("delete"))
        {
           deleteData();
           out.println("<br> data deleted successfully");
        }
    }
    public String insertData()
    {
try
        {
           Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
           Connection
con=DriverManager.getConnection("jdbc:odbc:TempDataSource","system","system");
           Statement st=con.createStatement();
           String query="insert into f_student values("+enr+",'"+name+"',"+sem+")";

           int count=st.executeUpDate(query);
           return "data inserted succesfully";
        }
catch(Exception e)
        {
          return e.toString();
        }
    }
    public void upDateData()
    {
try
        {
```

```
            Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
            Connection
con=DriverManager.getConnection("jdbc:odbc:TempDataSource","system","system");
            Statement st=con.createStatement();
            String query="upDate f_student set name='"+name+"' where enr="+enr+"";
            int count=st.executeUpDate(query);

        }catch(Exception e)
        {

          System.out.print(e);

        }
    }
    public void deleteData()
    {  try
        {
            Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
            Connection
con=DriverManager.getConnection("jdbc:odbc:TempDataSource","system","system");
            Statement st=con.createStatement();
            String query="delete from f_student where enr="+enr+"";
            int count=st.executeUpDate(query);

        }catch(Exception e)
        {

          System.out.print(e);

        }
    }

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        processRequest(request, response);
    }

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        processRequest(request, response);
    }

    @Override
```

```
    public String getServletInfo() {
        return "Short description";
    }// </editor-fold>
}
```

OUTPUT:





Logical Applications:

1. How we can insert the record in the database by using servlet?

   _____

   _____

   _____

2. How we can upDate the record in the database by using servlet?

   _____

   _____

   _____

Date:

PRACTICAL - 8

Cookies

Aim: Write a program to handle cookies.

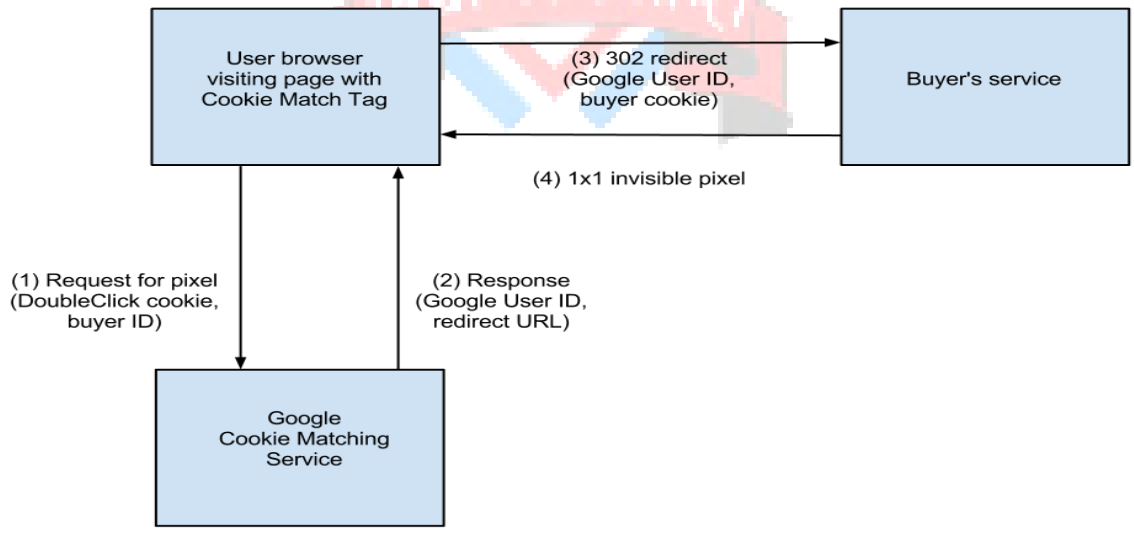
Theory: Cookie

- Cookie is used to transmit identifier between client and server.

Syntax:

```
Cookie co =new Cookie("name","XYZ");
response.addCookie(co);
```

Flowchart/Figure/Algorithm:



ss

Fig 2. Flowchart for Cookie

Program/Implementation:

SessionCookies.jsp

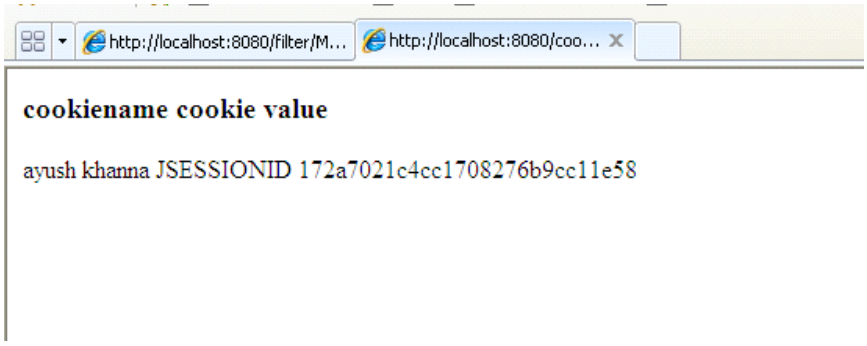
```
<%--
   Document   : SessionCookies
   Created on : Feb 19, 2016, 11:57:27 AM
   Author     : faraahdabhoiwala
--%>

<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
   <head>
      <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
      <title>JSP Page</title>
   <a href="SessionCookies">Retrive My Cookies</a>

   </head>
   <body>
      <h1>Hello World!</h1>
   </body>
</html>
```

SessionCookies.java

```
import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.http.Cookie;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import javax.servlet.http.HttpSession;

public class SessionCookies extends HttpServlet {

   protected void processRequest(HttpServletRequest request, HttpServletResponse response)
       throws ServletException, IOException {
      response.setContentType("text/html;charset=UTF-8");
      PrintWriter out = response.getWriter();

       Cookie c1=new Cookie("username","faraah");
      Cookie c2=new Cookie("password","f1234");
```

```
        response.addCookie(c1);
        response.addCookie(c2);
        Cookie[] c=request.getCookies();
        for(int i=0;i<c.length;i++)
        {
            out.println("<br>name of cookie is"+c[i].getName()+"value="+c[i].getValue());
        }

    }

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        processRequest(request, response);
    }
    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        processRequest(request, response);
    }

    /**
     * Returns a short description of the servlet.
     * @return a String containing servlet description
     */
    @Override
    public String getServletInfo() {
        return "Short description";
    }// </editor-fold>
}
```

OUTPUT:



Logical Applications*:*

1. To add the cookie in the browser, which method is used?

   _____
   _____
   _____

2. .How to retrieve the cookie value on browser?

   _____
   _____
   _____

Date:

PRACTICAL - 9

Filter

Aim: Write a program to create filter which does pre and post processing.

Theory:Filter

- Filter is a java class which performs pre and post processing.

Syntax:

```
public void init(FilterConfig filterConfig)
```

Flowchart/Figure/Algorithm:

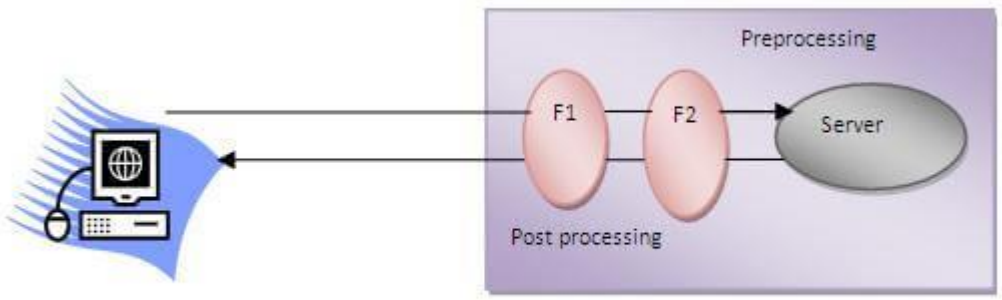


Fig 2. Flowchart for Filter

Program/Implementation:

newhtml.html

```
<!DOCTYPE html>
<html>
   <head>
      <title></title>
      <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
   </head>
   <body>
      <div>TODO write content</div>
      <a href="testServlet">go to my filter</a>
   </body>
</html>
```

MyFilter.java

```
import java.io.IOException;
import java.io.PrintStream;
import java.io.PrintWriter;
import java.io.StringWriter;
import javax.servlet.Filter;
import javax.servlet.FilterChain;
import javax.servlet.FilterConfig;
import javax.servlet.ServletException;
import javax.servlet.ServletRequest;
import javax.servlet.ServletResponse;

public class MyFilter implements Filter {

    private static final boolean debug = true;

    private FilterConfig filterConfig = null;

    public MyFilter() {
    }
    private void doBeforeProcessing(ServletRequest request, ServletResponse response)
            throws IOException, ServletException {
        if (debug) {
            log("MyFilter:DoBeforeProcessing");
        }
    }
```

```
private void doAfterProcessing(ServletRequest request, ServletResponse response)
        throws IOException, ServletException {
    if (debug) {
        log("MyFilter:DoAfterProcessing");
    }
}
public void doFilter(ServletRequest request, ServletResponse response,
        FilterChain chain)
        throws IOException, ServletException {

    if (debug) {
        log("MyFilter:doFilter()");
    }
    doBeforeProcessing(request, response);
    PrintWriter out=response.getWriter();
    out.print("<br>hi from filter");
    Throwable problem = null;
    try {
        chain.doFilter(request, response);
    } catch (Throwable t) {
        problem = t;
        t.printStackTrace();
    }
    doAfterProcessing(request, response);
    out.print("<br> after processing bye from filter");
        if (problem != null) {
        if (problem instanceof ServletException) {
            throw (ServletException) problem;
        }
        if (problem instanceof IOException) {
            throw (IOException) problem;
        }
        sendProcessingError(problem, response);
    }
}
    public FilterConfig getFilterConfig() {
    return (this.filterConfig);
}
public void setFilterConfig(FilterConfig filterConfig) {
    this.filterConfig = filterConfig;
}
public void destroy() {
}
public void init(FilterConfig filterConfig) {
    this.filterConfig = filterConfig;
    if (filterConfig != null) {
```

```
        if (debug) {
            log("MyFilter:Initializing filter");
        }
    }
}
@Override
public String toString() {
    if (filterConfig == null) {
        return ("MyFilter()");
    }
    StringBuffer sb = new StringBuffer("MyFilter(");
    sb.append(filterConfig);
    sb.append(")");
    return (sb.toString());
}
private void sendProcessingError(Throwable t, ServletResponse response) {
    String stackTrace = getStackTrace(t);

    if (stackTrace != null && !stackTrace.equals("")) {
        try {
            response.setContentType("text/html");
            PrintStream ps = new PrintStream(response.getOutputStream());
            PrintWriter pw = new PrintWriter(ps);
            pw.print("<html>\n<head>\n<title>Error</title>\n</head>\n<body>\n"); //NOI18N

            pw.print("<h1>The resource did not process correctly</h1>\n<pre>\n");
            pw.print(stackTrace);
            pw.print("</pre></body>\n</html>"); //NOI18N
            pw.close();
            ps.close();
            response.getOutputStream().close();
        } catch (Exception ex) {
        }
    } else {
        try {
            PrintStream ps = new PrintStream(response.getOutputStream());
            t.printStackTrace(ps);
            ps.close();
            response.getOutputStream().close();
        } catch (Exception ex) {
        }
    }
}
public static String getStackTrace(Throwable t) {
    String stackTrace = null;
```

```
        try {
            StringWriter sw = new StringWriter();
            PrintWriter pw = new PrintWriter(sw);
            t.printStackTrace(pw);
            pw.close();
            sw.close();
            stackTrace = sw.getBuffer().toString();
        } catch (Exception ex) {
        }
        return stackTrace;

    public void log(String msg) {
        filterConfig.getServletContext().log(msg);
    }
}
```

MyServlet.java

```
import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;

public class MyServlet extends HttpServlet {

        protected void processRequest(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        response.setContentType("text/html;charset=UTF-8");
        PrintWriter out = response.getWriter();
        out.print("<br>hello from servlet");
    }

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        processRequest(request, response);
    }
    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        processRequest(request, response);
    }
    @Override
    public String getServletInfo() {
        return "Short description";
```

```
    }// </editor-fold>
}
```

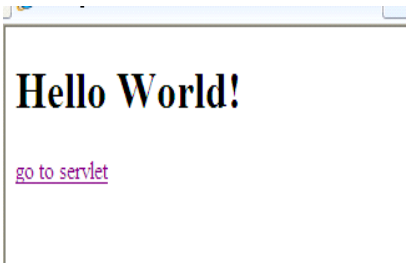
WEB-INF

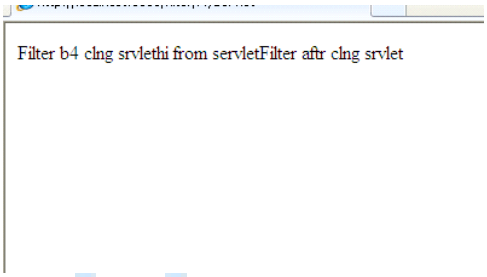
```
<?xml version="1.0" encoding="UTF-8"?>
<web-app version="3.0" xmlns="http://java.sun.com/xml/ns/javaee"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://java.sun.com/xml/ns/javaee http://java.sun.com/xml/ns/javaee/web-
app_3_0.xsd">
    <context-param>
        <param-name>admin</param-name>
        <param-value>khyati</param-value>
    </context-param>
    <context-param>
        <param-name>user</param-name>
        <param-value>sunny</param-value>
    </context-param>
    <filter>
        <filter-name>MyFilter</filter-name>
        <filter-class>MyFilter</filter-class>
    </filter>
    <filter>
        <filter-name>StudentFilter</filter-name>
        <filter-class>StudentFilter</filter-class>
    </filter>
    <filter-mapping>
        <filter-name>StudentFilter</filter-name>
        <servlet-name>StudentServlet</servlet-name>
    </filter-mapping>
    <servlet>
        <servlet-name>StudentServlet</servlet-name>
        <servlet-class>StudentServlet</servlet-class>
    </servlet>

    <filter-mapping>
        <filter-name>MyFilter</filter-name>
        <servlet-name>MyServlet</servlet-name>
    </filter-mapping>
    <servlet>
        <servlet-name>RetriveContextParam</servlet-name>
        <servlet-class>RetriveContextParam</servlet-class>
        <init-param>
            <param-name>country</param-name>
            <param-value>india</param-value>
        </init-param>
```

```
    <init-param>
      <param-name>state</param-name>
      <param-value>gujarat</param-value>
    </init-param>
</servlet>
<servlet>
    <servlet-name>SessionCookies</servlet-name>
    <servlet-class>SessionCookies</servlet-class>
</servlet>
<servlet>
    <servlet-name>testcookie</servlet-name>
    <servlet-class>testcookie</servlet-class>
</servlet>
<servlet>
    <servlet-name>URLRewriting</servlet-name>
    <servlet-class>URLRewriting</servlet-class>
</servlet>
<servlet>
    <servlet-name>JDBCServlet</servlet-name>
    <servlet-class>JDBCServlet</servlet-class>
</servlet>
<servlet>
    <servlet-name>MyServlet</servlet-name>
    <servlet-class>MyServlet</servlet-class>
</servlet>

<servlet-mapping>
    <servlet-name>RetriveContextParam</servlet-name>
    <url-pattern>/RetriveContextParam</url-pattern>
</servlet-mapping>
<servlet-mapping>
    <servlet-name>SessionCookies</servlet-name>
    <url-pattern>/SessionCookies</url-pattern>
</servlet-mapping>
<servlet-mapping>
    <servlet-name>testcookie</servlet-name>
    <url-pattern>/testcookie</url-pattern>
</servlet-mapping>
<servlet-mapping>
    <servlet-name>URLRewriting</servlet-name>
    <url-pattern>/URLRewriting</url-pattern>
</servlet-mapping>
<servlet-mapping>
    <servlet-name>MyServlet</servlet-name>
    <url-pattern>/MyServlet</url-pattern>
</servlet-mapping>
```

```
  <session-config>
    <session-timeout>30</session-timeout>
  </session-config>
</web-app>
```

OUTPUT:





Logical Applications:

1. What is the use of filterchain interface?

   _____
   _____
   _____

2. which are the applications of filter?

   _____
   _____
   _____

Date:

PRACTICAL - 10

useBean

Aim: **Write a program to implement useBean action tag.**

Source Code:

JBeanEmpInfo.jsp

```
<%--

Document:
JBeanAddEmpInfo Created on
: Feb 25, 2016, 8:59:56 AM
Author :
faraahdabhoiwala
--%>

<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
    <head>
        <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
        <title>JSP Page</title>
    </head>
    <body>

        <form method="post"
        action="JBeanViewEmpDetails.jsp"> Employee Code
        :<input type="text" name="empCode">                    </input>

        Employee Name :<input type="text" name="empName"></input>

        <input type="submit" value="submit">

        </input>
        </form>

    </body>
</html>
```

JBeanEmpDetails.jsp

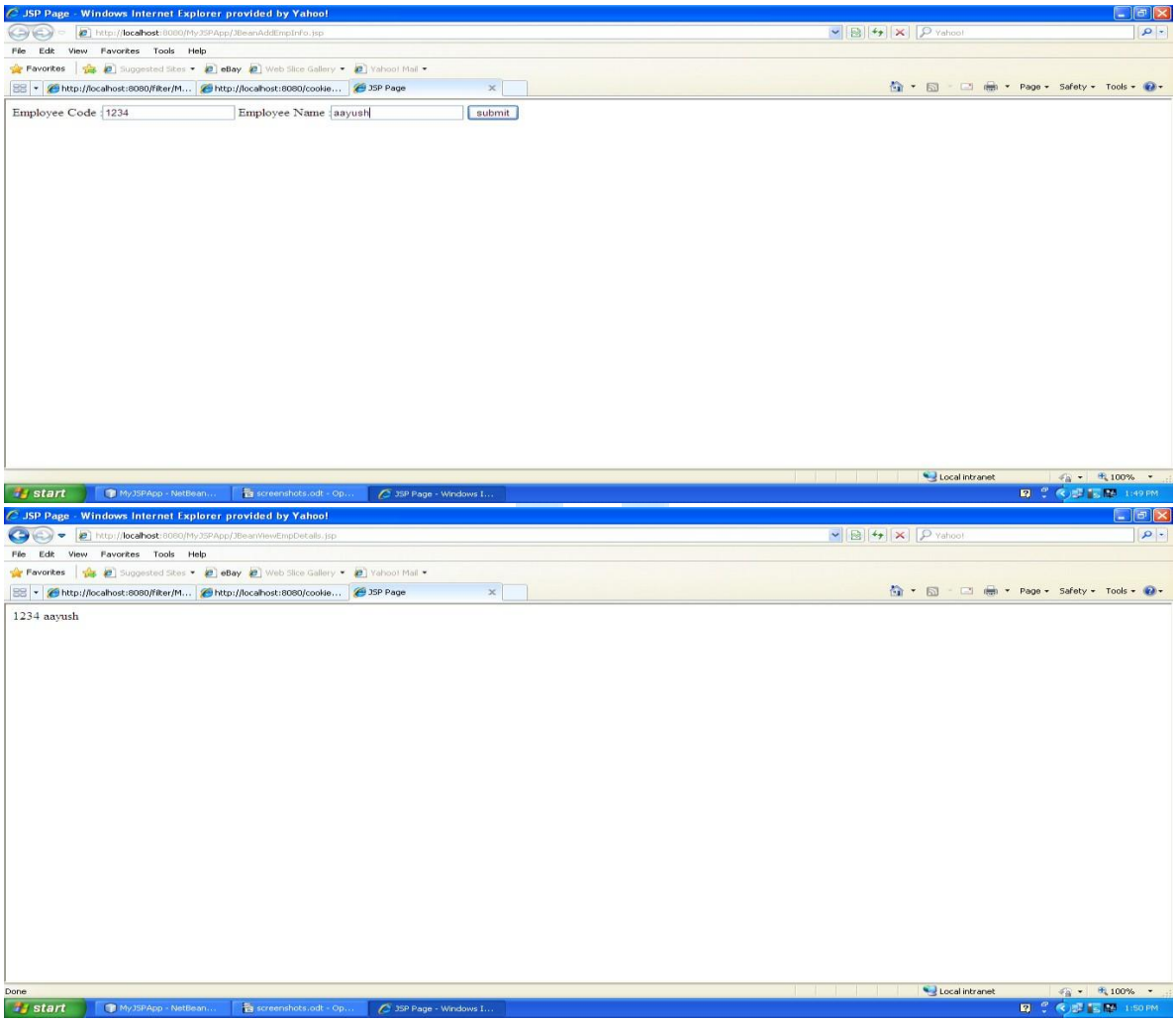
```
<%--
```

```
    Document   :
    JBeanViewEmpDetails
    Created on : Feb 25, 2016,
    9:08:06 AM Author      :
    faraahdabhoiwala
--%>

<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>

<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<title>JSP Page</title>
</head>
    <body>
      <jsp:useBean id="emp"  class="p1.JBean" />
      <jsp:setProperty name="emp" property="*" />
    <jsp:getProperty name="emp" property="empCode" />
     <jsp:getProperty name="emp" property="empName" />
 </body>
</html>
```

JBean.java

```
package p1;
 import java.io.*;
public class JBean implements Serializable {
 private String empCode;
private String empName;
public String getEmpName()
{
return empName;
}
public void setEmpName(String empName)
{
this.empName=empName;
}
public String getEmpCode()
{
return empCode;
}
public void setEmpCode(String empCode)
{
this.empCode=empCode;
}
}
```

OUTPUT:



Logical Applications**:**

1. What is the use of Serializable interface?

_____

_____

_____

2. which are use os useBean action tag?

_____

_____

_____

Date:

PRACTICAL – 11

JSP page

Aim: **Write a program to create a Custom tag and use it in a JSP page.**

Source Code:

MyTagHandler.ja

va

```
package p1;
import java.io.IOException;
 import javax.servlet.jsp.JspWriter;
import javax.servlet.jsp.JspException;
 import javax.servlet.jsp.tagext.JspFragment;
import javax.servlet.jsp.tagext.SimpleTagSupport;

public class MyTagHandler extends SimpleTagSupport {
@Override
public void doTag() throws JspException ,IOException{
JspWriter out = getJspContext().getOut();
out.print("hello everyone this is custom tag");
}
}
```

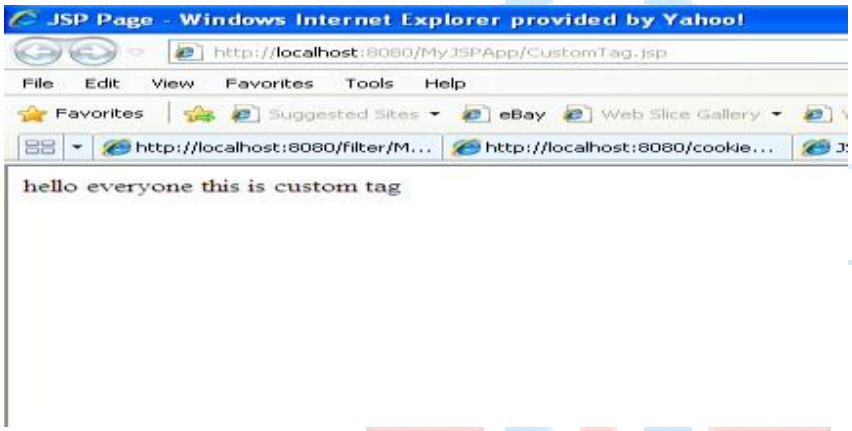
Mytag_libraray.tld

```
<?xml version="1.0" encoding="UTF-8"?>
<taglib version="2.1" xmlns="http://java.sun.com/xml/ns/javaee"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://java.sun.com/xml/ns/javaee http://java.sun.com/xml/ns/javaee/web-
jsptaglibrary_2_1.xsd">
<tlib-version>1.0</tlib-version>
<short-name>mytag_library</short-name>
<uri>/WEB-INF/tlds/Mytag_library</uri>
<tag>
<name>Display</name>
<tag-class>p1.MyTagHandler</tag-class>
<body-content>scriptless</body-content>
</tag>
</taglib>
```

CustomTag.java

```
<%@taglib prefix="f" uri="/WEB-INF/tlds/Mytag_library" %>
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
  <html>
    <head>
      <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
      <title>JSP Page</title>
    </head>
    <body>
      <f:Display></f:Display>
    </body>
  </html>
```

OUTPUT:



Logical Applications:

1. **Who catch the exceptions and which type of exceptions?**

   _____

   _____

   _____

2. **Which are the use of custom tag?**

   _____

   _____

   _____

Date:

PRACTICAL – 12

Aim: **Write a program using connectionless protocol in which the client sends a string to server and server converts the string into uppercase and sends it back to client**

Source Code:

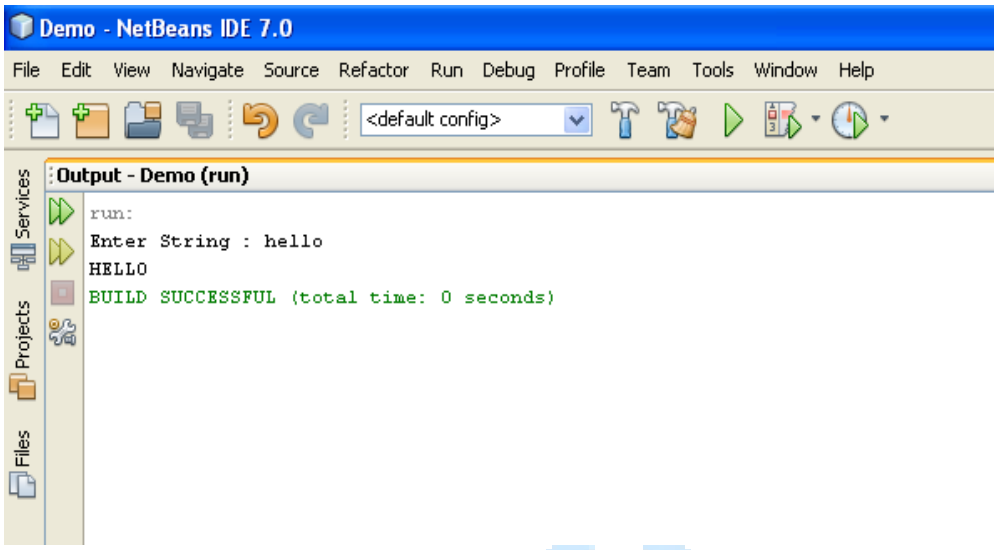
DatagramServer.java

```
package mynetworkapp;
import java.net.*;
import java.io.*;

public class DatagramServer {

public static void main(String[] args) {
try{
DatagramSocket ds=new DatagramSocket(2000);//port no of socket--Socket Exception
byte b[]=new byte[100];
DatagramPacket dp=new DatagramPacket(b,b.length);//blank packet
ds.receive(dp);
b=dp.getData
String d=new String(b).trim();//convert to string
System.out.println(d);
//task to be performed on string//
String d1=d.toUpperCase();//task
System.out.println(d1);
byte b1[]=new byte[100];//byte array for sending data
b1=d1.getBytes();//convert string form data to bytes
DatagramPacket sp=new
DatagramPacket(b1,b1.length,dp.getAddress(),dp.getPort());
ds.send(sp);//send the packet to client--throws IOException
}
 catch (Exception e) {
System.out.println(e);
    }

    }

}
```

DatagramClient.java

```
package mynetworkapp;

import java.net.*;  import java.util.Scanner;

public class DatagramClient {

public static void main(String[] args) {

 try {
DatagramSocket ds=new DatagramSocket();//thows SocketException
Scanner sc=new Scanner(System.in);
System.out.print("Enter a String :");
String str=sc.next();

byte b[]=new byte[100];
b=str.getBytes();
//create a packet and store the data in it
DatagramPacket dp=new DatagramPacket(b,b.length,InetAddress.getLocalHost(),2000);
//send the packet
ds.send(dp);

byte b1[]=new byte[100];
//create an empty packet to receive the data
DatagramPacket rp=new
DatagramPacket(b1,b1.length);
//fill the packet
ds.receive(rp);
//unpack the data in byte form
b1=rp.getData();
String d=new
String(b1).trim();
System.out.println(d);

        } catch (Exception e) {

            System.out.println(e);
        }

    }

  }
```

OUTPUT:



Logical Applications**:**

1.What is connectionless protocol?

_____

_____

_____

2.Name of connectionless protocol.

_____

_____

_____

Date:

PRACTICAL – 13

Aim: **Write a JSF application to create a login form.**

Source Code:
LoginBean.java

```
import java.io.Serializable;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import javax.faces.bean.ManagedBean;
import javax.faces.bean.SessionScoped;

@ManagedBean(name = "loginBean")

public class LoginBean implements Serializable{
String userName;
String password;
public String getPassword() {
 return password;
}
public void setPassword(String password) {
this.password = password;
}
public String getUserName() {
return userName;
}
public void setUserName(String userName) {
 this.userName = userName;
}
public String checkValidUser()
{
try{
Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
Connection
con=DriverManager.getConnection("jdbc:odbc:TempDataSource","system","system");
Statement st=con.createStatement();

ResultSet rs=st.executeQuery("select * from user1 where username='"+userName+"' and
password='"+password+"'");
rs.next();
```

```
if(rs.getRow()==1)
{
return "success";
}
else
{
return "failure";
}
}catch(Exception e)
{
return e.toString();
}
}
public LoginBean() {
}

}
```

login.jsp

```
<%--
Document   : login
Created on : Mar 15, 2016, 9:33:42 AM
Author : faraahdabhoiwala
--%>
<%@taglib uri="http://java.sun.com/jsf/core" prefix="f"%>
<%@taglib uri="http://java.sun.com/jsf/html" prefix="h"%>
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<f:view>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<title>JSP Page</title>
</head>
<h1>Hello World!</h1>
<body>
<h:form>
Enter UserName :<h:inputText id="userName" value="#{loginBean.userName}"><br/>

</h:inputText>
Enter Password : <h:inputText id="password" value="#{loginBean.password}"/><br/>
<h:commandButton  value="login" action="#{loginBean.checkValidUser()}"/>
</h:form>

</body>
```

```
</html>
</f:view>
```

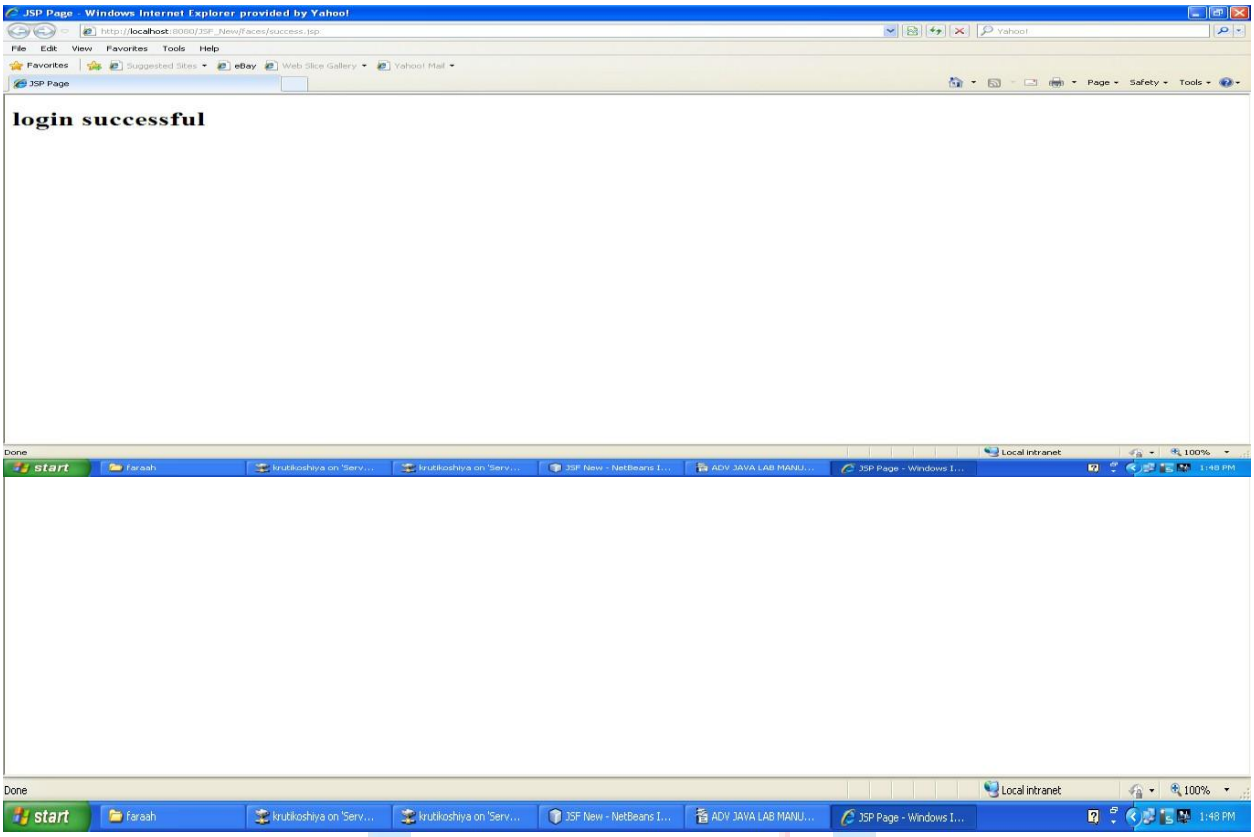
success.jsp

```
<%--

    Document   : success
    Created on : Mar 15, 2016,
    9:21:09 AM Author        :
    faraahdabhoiwala
--%>

<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>

<html>
    <head>
        <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
        <title>JSP Page</title>
    </head>
    <body>
          <h1>login successful</h1>
    </body>
</html>
```

failure.jsp

```
<%--

    Document   : failure
    Created on : Mar 15, 2016,
    9:21:21 AM Author        :
    faraahdabhoiwala
--%>

<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
    <head>
        <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
        <title>JSP Page</title>
    </head>
    <body>
        <h1>invalid username/pwd <br/>login failed</h1>
    </body>
</html>
```

OUTPUT:



Logical Applications:

1. What is the use of JSF?

_____

_____

_____

2. Which libraries required for JSF?

_____

_____

_____

Date:

PRACTICAL – 14

Aim: **Write a JSF application to insert, up**Date **and delete record from database.**

Source Code:

LoginBean.java

```
import java.io.Serializable;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import javax.faces.bean.ManagedBean;
import javax.faces.bean.SessionScoped;

@ManagedBean(name = "loginBean")
//@Dependent
 @SessionScoped
public class LoginBean implements Serializable{
String userName;
String password;
public String getPassword() {
return password;
}
public void setPassword(String password) {
 this.password = password;
}
public String getUserName() {
return userName;
}
public void setUserName(String userName) {
this.userName = userName;
}
public String checkValidUser()
{
try
{
Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
Connection
con=DriverManager.getConnection("jdbc:odbc:TempDataSource","system","system");
```

```
Statement st=con.createStatement();

ResultSet rs=st.executeQuery("select * from user1 where username='"+userName+"' and
password='"+password+"'");
rs.next();
if(rs.getRow()==1)
{
return "success";
}
else
{
return "failure";
}
}catch(Exception e)
{
return e.toString();
}

}

public LoginBean() {
}
}
```

login.jsp

```
<%--
Document   : login
Created on : Mar 15, 2016, 9:33:42 AM
Author : faraahdabhoiwala
--%>
<%@taglib uri="http://java.sun.com/jsf/core" prefix="f"%>
<%@taglib uri="http://java.sun.com/jsf/html" prefix="h"%>
<%@page contentType="text/html" pageEncoding="UTF-8"%>

<!DOCTYPE html>
<f:view>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<title>JSP Page</title>
</head>
<h1>Hello World!</h1>
<body>
<h:form>
Enter UserName :<h:inputText id="userName" value="#{loginBean.userName}"><br/>
```

```
</h:inputText>
Enter Password : <h:inputText id="password" value="#{loginBean.password}"/><br/>
<h:commandButton  value="login" action="#{loginBean.checkValidUser()}"/>
</h:form>
</body>
</html>
</f:view>
```

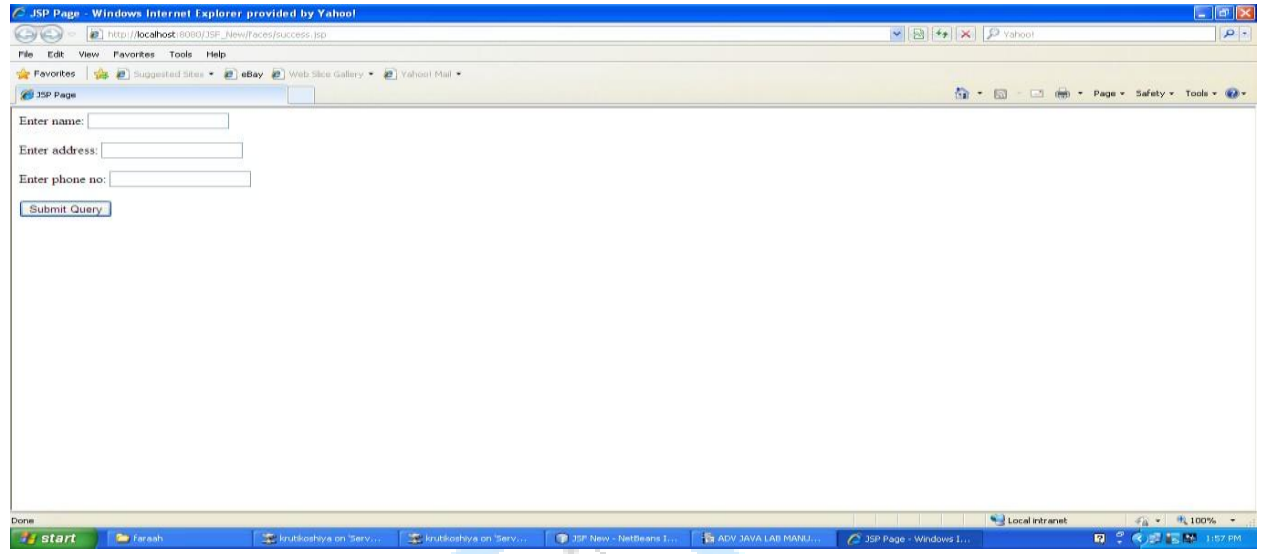
success.jsp

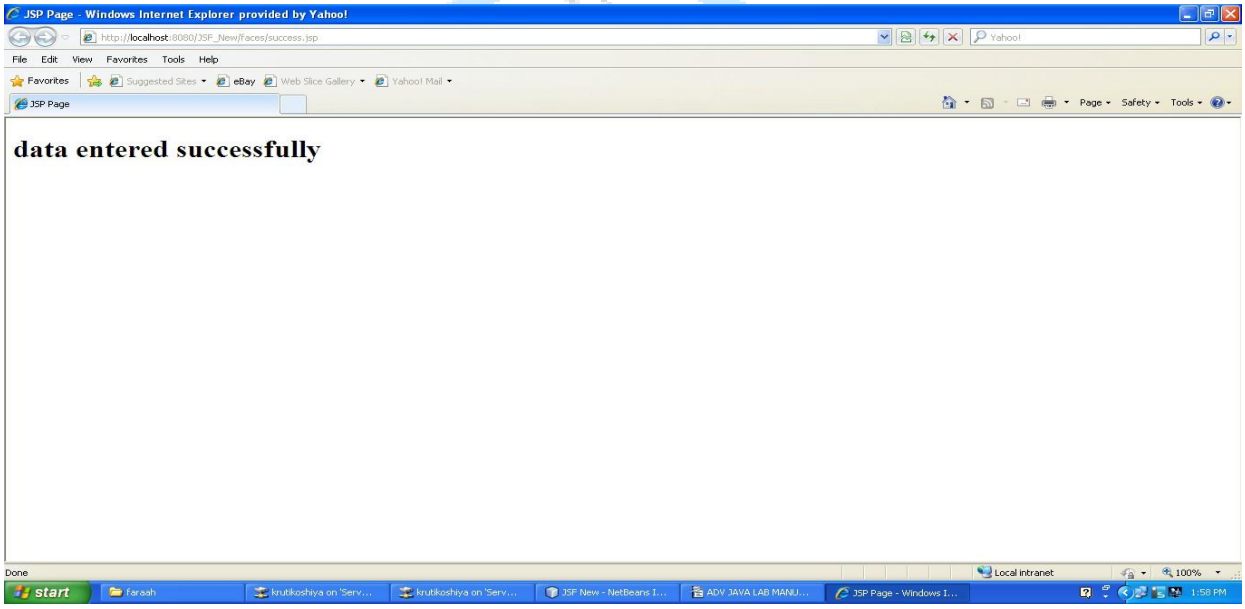
```
<%--
Document   : success
Created on : Mar 15, 2016, 9:21:09 AM
Author : faraahdabhoiwala
--%>
<%@page contentType="text/html" pageEncoding="UTF-8"%>

<!DOCTYPE html>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<title>JSP Page</title>
</head>
<body>
<h1>login successful</h1>
</body>
</html>
```

failure.jsp

```
<%--
Document   : failure
Created on : Mar 15, 2016, 9:21:21 AM
Author : faraahdabhoiwala
--%>
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<title>JSP Page</title>
</head>
<body>
<h1>invalid username/pwd <br/>login failed</h1>
</body>
</html>
```

OUTPUT:


JSP Page - Windows Internet Explorer provided by Yahoo!
Enter name:
Enter address:
Enter phone no:
Submit Query


JSP Page - Windows Internet Explorer provided by Yahoo!
data entered successfully

Logical Applications**:**

1. How JSF application connect with Database?

_____
_____
_____

2. How data retrive from the database?

_____
_____
_____